

Sistema de Gestión de Caja

Materia: Metodologías de Software (MDSW)

NRC 30693.

Docente: Ing. Jenny Ruiz.

Equipo: Grupo #7

Leopoldo Bolaños, Matias Medina, Martin Palacios.





Contexto de la problemática

- En los pequeños negocios que ofrecen productos y servicios, el control de las ventas diarias, los métodos de pago y el cierre de caja suele realizarse de forma manual. Esta situación genera pérdida de tiempo, errores de cálculo y dificultad para conocer con exactitud cuánto dinero ingresó por efectivo, transferencia, tarjeta o pago mixto.
- Por esta razón, se propone desarrollar una aplicación de escritorio en Java que permita registrar productos de tienda, servicios TAG, servicios variables, clientes, métodos de pago, caja chica y cierres de caja. El sistema permitirá generar un reporte final de jornada con subtotales, desglose por método de pago, caja chica y gran total.
- Con esta aplicación, el administrador del negocio podrá reducir el tiempo dedicado al registro manual de ventas y al cálculo del cierre diario de caja. El sistema permitirá centralizar la información de productos, servicios, clientes, métodos de pago y caja chica en una aplicación c



Objetivos

Objetivo General:

Desarrollar una aplicación de escritorio en Java que permita automatizar el registro de productos y servicios para el cierre de caja de un negocio, utilizando el patrón Modelo-Vista Controlador y una base de datos no relacional para mejorar el control de los movimientos diarios y reducir errores en el proceso administrativo.

Objetivos Específicos:

1. Identificar los requisitos funcionales del sistema mediante una matriz de historias de usuario basada en la técnica 5W+2H.
2. Diseñar e implementar los módulos principales del sistema: acceso, productos, cartera de clientes, servicios TAG, servicios variables, método de pago, caja chica, cierre de caja e historial de cierres.
3. Validar el funcionamiento del sistema mediante pruebas de caja negra y caja blanca, verificando que los módulos desarrollados cumplan con los requisitos definidos.

</> Entorno de desarrollo

Metodología Ágil (Scrum, Jira y Confluence): El trabajo se organizó bajo el marco Scrum, dividiendo el proyecto en sprints, requisitos funcionales e historias de usuario (5W+2H). Se utilizó Jira para planificar tareas y gestionar el backlog, y Confluence para centralizar la documentación técnica.



Arquitectura y Tecnologías: La aplicación se desarrollará en Java usando Apache NetBeans, aplicando Programación Orientada a Objetos y el patrón Modelo-Vista-Controlador (MVC). Además, se implementará MongoDB como base de datos para almacenar la información de los clientes y los cierres de caja.



Desarrollo Colaborativo (GitHub): Se empleó como herramienta de control de versiones. Permitió a los integrantes del equipo programar simultáneamente, unificar los módulos de manera segura y mantener un historial detallado de los cambios en el código.



MARCO TEÓRICO - METODOLOGÍA

(Marco de trabajo 5W+2H)

¿QUÉ?

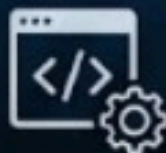
(Proyecto)



- Desarrollar una aplicación de escritorio.
- Gestión de productos, servicios (TAG, variables), clientes.
- Registro de métodos de pago, caja chica, cierres diarios.

¿CÓMO?

(Tecnología)



- Construido en Java.
- Arquitectura POO y Patrón MVC.
- Base de datos MongoDB.
- Metodología Scrum (Sprints, historias de usuario).

¿QUIÉN?

(Equipo)



- Estudiantes del Grupo 7: Nombres de los estudiantes (e.g., Bolaños Leopoldo, Palacios Martín, Medina Matías).
- Superv. académica: Ing. Jenny Ruiz. Tutora empres.: Tuto: Lic. Jessica Campoverde.
- Usuario: Administrador del negocio.

¿CUÁNDO?

(Cronograma)



- Durante el semestre académico.
- Entregas parciales por sprints.
- Presentación final al término del periodo.

¿POR QUÉ?

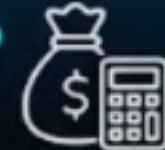
(Justificación)



- Automatizar control de ventas, clientes, caja manual.
- Reducir pérdida de tiempo, errores de cálculo.
- Facilitar consulta de información histórica.
- Mejorar control administrativo.

¿CUÁNTO?

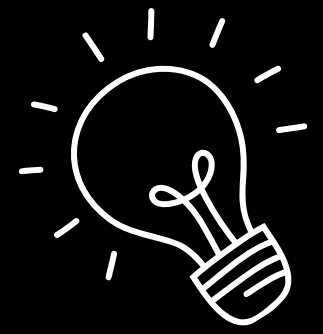
(Presupuesto)



- Costo de equipos (laptops).
- Licencias de software necesarias.
- Total Aproximado: 3645 USD.



100% COMPLETE
% DE CUMPLIMIENTO DEL PROYECTO



Ideas para Defender

1. Demostrar que la programación orientada a objetos permite construir un sistema modular, organizado y fácil de mantener, separando las responsabilidades del programa en clases relacionadas con productos, servicios, clientes, métodos de pago y cierre de caja.
2. Evidenciar que el uso del patrón Modelo-Vista-Controlador facilita la organización del código, ya que separa la interfaz gráfica, la lógica de negocio y el control de eventos del sistema.
3. Defender que una aplicación de escritorio con almacenamiento en MongoDB puede mejorar el control administrativo de un negocio pequeño, reduciendo errores en el registro de ventas, métodos de pago, caja chica y cierres diarios.
4. Mostrar que el uso de Scrum, historias de usuario y pruebas de software permite organizar el desarrollo por requisitos funcionales y validar que cada módulo cumpla con las necesidades del usuario final.

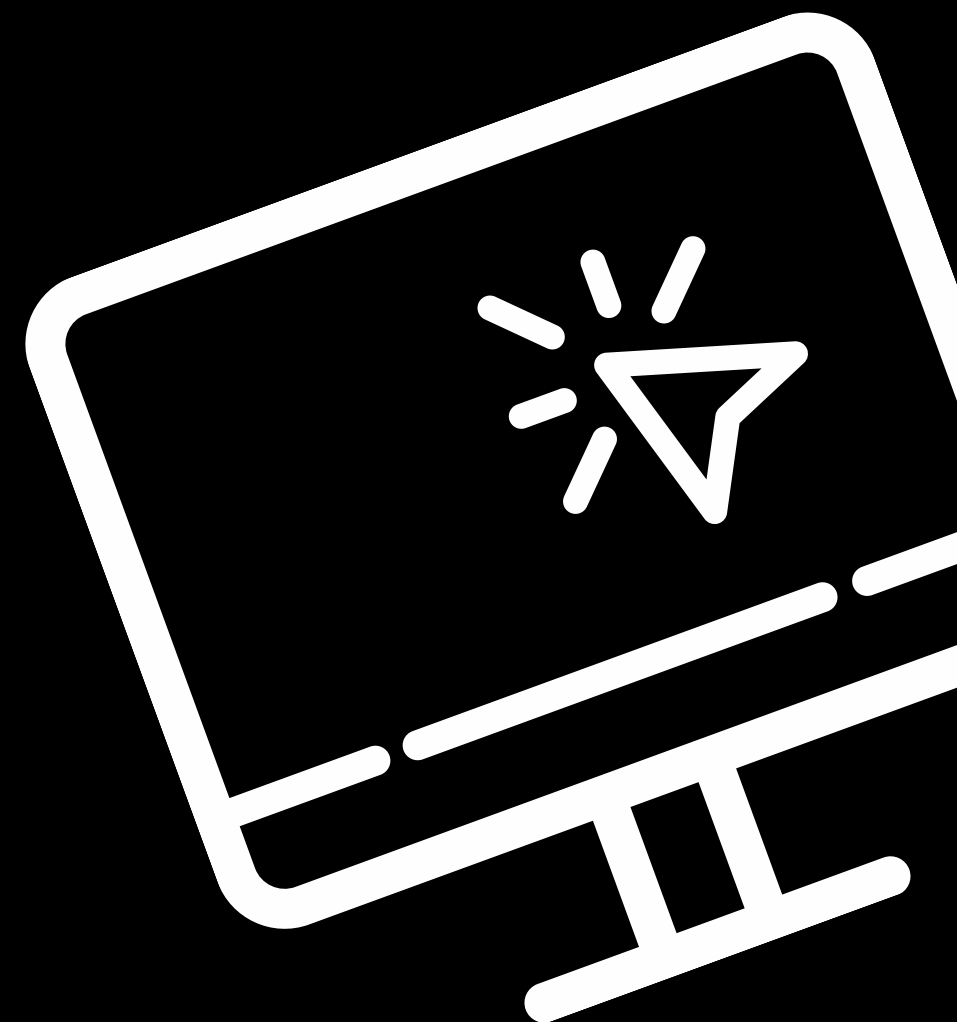




Resultados Esperados

- Una aplicación de escritorio funcional que permita al administrador registrar productos de tienda, servicios TAG, servicios variables, clientes, métodos de pago, caja chica y cierres de caja.
- Un módulo de cierre de caja que genere automáticamente un reporte final de jornada con subtotales, desglose por método de pago, caja chica y gran total.
- Un sistema de inicio de sesión que limite el acceso únicamente al administrador, protegiendo la información registrada en la aplicación.
- Una cartera de clientes que permita registrar cédula, nombres, celular y correo electrónico, además de buscar clientes por cédula y mostrar todos los registros guardados.
- Un módulo de historial que permita consultar cierres de caja anteriores mediante búsqueda por fecha o visualización completa de los reportes guardados.
- Un sistema conectado a MongoDB para almacenar información relevante, como clientes y cierres de caja, permitiendo conservar los datos, aunque la aplicación se cierre.
- Un código organizado bajo el patrón Modelo-Vista-Controlador, facilitando su mantenimiento, revisión y futuras mejoras.

Viabilidad



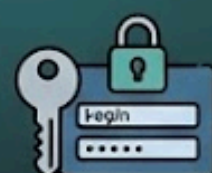
Cantida d	Descripción	Valor Unitario (USD)	Valor Total(USD)
	Equipo en casa		
1	HP VICTUS 16.1 R7 8845hs 8 núcleos 16 hilos / 32gb RAM 5600Mhz DDR5 /1 Tb SSD / GPU: RTX 4070 8GB GDDR6	1500	1500
1	Modelo: GIGABYTE G5 KF » Procesador: Intel Core™ i5-12500H 4,5 GHz (12 núcleos/16 subprocesos, 18 MB de caché) » RAM: 16GB (2×8) DDR4 (DDR4-3200, máx. 64 GB) » Almacenamiento: 512GB M.2 2280 » GPU: NVIDIA® GeForce RTX™ 4060 8GB GDDR6	1000	1000
1	Dell G15 5530 con un Intel Core i7-13650HX (13th Gen) 16 GB de RAM DDR5 (4800 MT/s) 8 GB de tarjeta gráfica (GPU dedicada) 954 GB de almacenamiento	1000	1000
	Software		
1	Sistema operativo Windows 11	145	145
1	Apache NetBeans IDE	0	0
		TOTAL	3645

Tabla 2 Presupuesto del proyecto



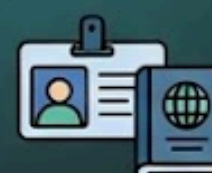
Alcance del Sistema

Resumen de Funciones Clave para el Administrador:



Control de Acceso

- Iniciar sesión mediante usuario y contraseña.



Gestión de Productos y Clientes

- Registrar productos de tienda con precios definidos.
- Registrar clientes (Cédula, Nombres, Celular, Correo).
- Buscar clientes y ver la cartera registrada.



Gestión de Servicios

- Registrar servicios TAG y servicios fijos mediante opciones predefinidas.
- Registrar servicios variables (concepto y valor manual).



Proceso de Venta y Pago

- Registrar método de pago (Efectivo, Transf., Tarjeta, Mixto).
- Validar que los montos de pago mixto coincidan con el total.



Operaciones de Caja

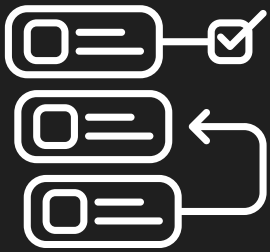
- Gestionar caja chica actual y para el día siguiente.
- Generar cierre final (Subtotales, Pagos, Caja Chica y Total).



Datos y Consultas



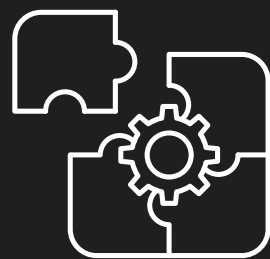
- Guardar los cierres de caja en MongoDB.
- Consultar cierres anteriores por fecha o ver todos.



Backlog

Nuestro backlog contiene los requisitos esenciales para el sistema de gestión de caja, priorizados para guiar el desarrollo. Esta tabla detalla cada funcionalidad, su descripción y nivel de prioridad, asegurando una hoja de ruta clara para las futuras iteraciones del proyecto.

ID	Módulo / Requisito	Descripción	Prioridad
REQ001	Control de acceso	Login seguro para administradores.	Alta
REQ002	Venta de Productos	Registro y venta de snacks.	Alta
REQ003	Registro de datos	Gestión de clientes/usuarios.	Alta
REQ004	Gestión de TAGs	Registro y pago de servicios de peaje.	Alta
REQ005	Precio Variable	Ingreso manual de servicios especiales.	Alta
REQ006	Reporte de Ventas	Cierre de caja con desglose de ganancias.	Alta
REQ007	Caja Chica	Registro de gastos menores y apertura.	Urgente
REQ008	Búsqueda Historial	Filtro por fecha de cierres anteriores.	Alta
REQ009	Métodos de Pago	Desglose y validación de pagos (Mixto).	Alta



Sistema de Gestión de Caja: Flujo Integral

Una solución diseñada para la Lic. Jessica Campoverde que profesionaliza el manejo de su negocio, asegurando la precisión contable de principio a fin.



Entrada de Datos

El sistema captura transacciones de venta de snacks (**REQ002**), servicios TAG (**REQ004**) y servicios variables (**REQ005**).



Validación Financiera

El corazón del sistema. Si un pago es "Mixto" (**REQ009**), no permite guardar la transacción hasta que la suma de todas las partes coincida con el total.



Salida y Persistencia

Guarda información en MongoDB, actualiza el stock, y permite generar reportes históricos y cierres de caja detallados (**REQ006**, **REQ008**).

Este enfoque garantiza:

Trazabilidad

Control preciso del flujo de cada centavo.

Integridad

Elimina errores de digitación en los pagos.

Auditoría

Facilita la búsqueda y verificación de cierres históricos.



Conclusiones

Se logro desarrollar una solución en forma de aplicación de escritorio en Java que automatizo todo el proceso del registro de productos y servicios para el cierre de caja de un negocio usando el Modelos Vista Controlador y una base de datos no relacional para mejorar el control de los movimientos diarios reduciendo los errores administrativos eficientemente.

- Se definieron con precisión las necesidades del sistema mediante la técnica 5W+2H y las historias de usuario, asegurando una base clara y sin ambigüedades para el desarrollo.
- Se construyeron con éxito todos los módulos clave del software (desde el acceso hasta la gestión de caja), logrando una plataforma operativa, integrada y funcional.
- Se construyeron con éxito todos los módulos clave del software (desde el acceso hasta la gestión de caja), logrando una plataforma operativa, integrada y funcional.



Recomendaciones

1. Se recomienda mejorar la comunicación entre el equipo de desarrollo y el cliente con el fin de comprender mejor las necesidades reales del negocio y evitar cambios tardíos en los requisitos funcionales.
2. Asignar roles precisos basándose en la habilidad de cada integrante del equipo es esencial para un flujo de trabajo organizado y continuo, además de brindar apoyo en cada situación imprescindible, en muchas ocasiones puede ocurrir que se tiene dificultades en algún punto específico del trabajo y ahí es cuando entra en acción las habilidades grupales de equipo de trabajo
3. Si otro equipo va a desarrollar un sistema similar, mi recomendación principal es que analicen con mucho cuidado el tiempo real que les tomará cada tarea desde la etapa de planificación. Además, para mantener el tablero ordenado y no saturarse, sugiero que terminen por completo una función antes de empezar a desarrollar la siguiente. Trabajar de esta manera evita que se acumule la carga para los últimos días y permite llevar el pro